

Supplementary Material — Software-as-a-Graph: Parameter Sensitivity of the Explanation Layer

Ibrahim Onuralp Yigit^a, Feza Buzluca^a

^aDepartment of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey

This supplement reports the parameter-sensitivity analyses supporting RQ3 of the main manuscript. They are placed here rather than in the body because they establish *robustness* of the reported orderings rather than any headline result: their collective finding is that the explanation layer’s ten declared weight constants are not load-bearing, with the two exceptions identified below. Section and table numbers are prefixed **S**. All figures are regenerable from the committed artifacts listed in the replication package.

S1. Parameter Sensitivity of the Explanation Layer

The RM composite of Section 5 of the main manuscript declares ten weight constants. We screen all of them, one factor at a time where a natural sweep range exists and jointly via Morris elementary effects [1, 2] otherwise, and report both views in Table S1. Morris is a screening design: μ^* ranks influence on mean ρ without decomposing it, and a σ comparable to a factor’s own μ^* signals that its effect depends on where the other nine sit rather than being a fixed marginal slope. It is the computationally efficient alternative to variance-based global analysis [3, 4].

Table S1: Sensitivity of the RM composite to its ten declared weight constants. **OFAT** reports one-factor-at-a-time sweeps against $I^*(v)$ over the seven core synthetic domains; **Morris** reports elementary-effects screening over six scenarios (10 trajectories, 110 evaluations), where μ^* is influence on mean ρ and σ its interaction spread. Rows are ordered by μ^* . Only r_α and λ are load-bearing.

Constant	One-factor sweep (OFAT)	Morris μ^*	Morris σ
r_α (Fault Tolerance / Availability blend)	not swept individually	0.132	0.040
λ (AHP shrinkage, uniform $\rightarrow$ raw)	$\rho = 0.319 \rightarrow 0.200$, monotone (spread 0.119)	0.134	0.090
w_{dur} (QoS durability)	part of the AHP-derived QoS vector (0.24, 0.62, 0.14); not swept separately	0.013	0.016
w_{prio} (QoS priority)		0.022	0.024
w_{rel} (QoS reliability)		0.012	0.018
α (topic payload size)	swept jointly over the full (β, α, ψ) simplex, 7 points: $w(t)$ ordering never falls below $\rho = 0.919$ against the shipped split; downstream spread not swept individually	0.019	0.026
β (topic QoS term)		0.025	0.028
ψ (topic frequency)		0.011	0.011
p (power-mean exponent)	not swept individually 0.031 (Topo-QoS), 0.007 (RM)	0.005	0.007
γ (library fan-out)		0.001	0.001

Three things follow. First, the parameter risk of the explanation layer is concentrated in two constants, both internal to the RM composite; the QoS-derived weights on which the framework’s architectural story rests are not load-bearing for its output. Second, the topic-weight split $(\beta, \alpha, \psi) = (0.75, 0.15, 0.10)$ is a documented convention rather than a tuned parameter — no choice within its simplex, including uniform weighting, would meaningfully change any result in this paper. Third, Dirichlet sampling over 100 draws of the whole weight vector confirms that the parameterization is stable as a whole: mean $\rho = 0.200$ (sd 0.010, 90% interval [0.185, 0.218]), with mean Kendall $\tau = 0.826$ and mean top-20% Jaccard 0.739 against the shipped ranking.

The elicited weights are anti-predictive, and we keep them anyway. The λ row deserves separate comment because its direction is unfavorable. Rank correlation falls monotonically as the intra-dimension weights move from a uniform prior toward the raw AHP judgment ($0.319 \rightarrow 0.200$, Figure S1): expert elicitation makes the ranking worse, by 0.119. We retain $\lambda = 0.70$ regardless, and the reasoning should be explicit rather than assumed. RM is an attribution instrument, not a ranking model (Section 6.2 of the main manuscript); its purpose is to say *why* a component is fragile in auditable, standards-referenced terms, and rank correlation against a cascade oracle is not the quantity it is built to maximize. Tuning λ toward zero would improve a number we do not claim at the cost of the traceability we do. What this sweep establishes is nonetheless a genuine limitation: we have evidence that the elicited weights produce worse rankings and none that they produce better attributions. Validating the attribution itself — against practitioner judgment, or against the outcome of applying the repairs it recommends — is outside this study and is the explanation layer’s principal open question (Section 8.4 of the main manuscript).

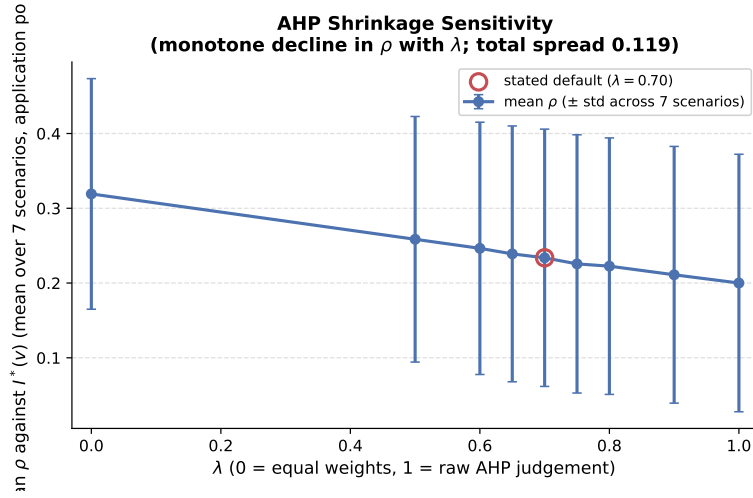


Figure S1: RM composite rank correlation against $I^*(v)$ across the AHP shrinkage parameter λ (Application population, mean over the seven core synthetic domains). The descent is monotone: the closer the intra-dimension weights move to raw elicited judgment, the worse the ranking. Rendered from `results/ahp_shrinkage_sweep_v3.json`.

S2. Zero-Inflation Sensitivity of the Agreement Figures

Spearman ρ over a population where many components are tied at exactly zero is driven substantially by how those ties are handled, and $I^*(v)$ is heavily zero-inflated: 19 to 106 Applications per scenario carry exactly zero cascade impact, whereas $I_{\text{comp}}(v)$ has no exact zeros at all. We therefore report, alongside each full-population figure, $\rho_{>0}$ — the same correlation restricted to components both oracles score strictly positive. It is a sensitivity bound, not a replacement: for a component the simulator actually injected, zero impact is a real measurement ("its failure reaches nobody"), and dropping it would be a results-favorable filter.

The bound changes the reading of two scenarios, in opposite directions. On `hub_and_spoke`, I_{comp} and I^* correlate at $\rho = -0.044$ over the full population but $\rho_{>0} = +0.263$ over the 22 components both score positive: the apparent *sign* disagreement is an artifact of tied zeros, not evidence that the oracles order active components inversely. On `microservices`, the movement runs the other way — $\rho = 0.461$ falls to $\rho_{>0} = 0.257$, so a substantial part of that scenario’s agreement is the two oracles concurring on which components are inert rather than on how the active ones rank. Averaged over the seven scenarios the two summaries are close ($\rho = 0.425$, $\rho_{>0} = 0.447$), which is precisely why the per-scenario figures matter: the mean conceals compensating movements of 0.2–0.3 in both directions.

S3. Domain-Specific Weighting and Threshold Sensitivity

Sweeping the composite reliability weight $w_R \in [0, 1]$ moves mean ρ by only 0.018 (0.317 at $w_R = 0$ to 0.336 at $w_R = 1$), and the domain-derived and static rankings agree at mean Kendall $\tau = 0.980$ — the two orderings are very nearly the same ordering. Within that narrow band the domain-derived weighting does not improve on either alternative it replaces: mean ρ is 0.318 (domain-derived) against 0.331 (equal) and 0.321 (static), so it is marginally the *worst* of the three, losing to equal weighting by 0.013 and to static by 0.003. Both margins are far smaller than the between-model differences of Section 7.1 of the main manuscript and we draw no ranking conclusion from either. What the sweep establishes is that no weighting confined to this parameter could move ρ appreciably, so ISO/IEC 25019 context-of-use reweighting must be understood as an attributional device that expresses criticality in stakeholder terms — and not, on this evidence, as a mechanism that makes the ranking more accurate. Two free parameters of the ground truth itself matter more than any scoring weight, and we sweep both. The cascade propagation threshold moves mean ρ across a spread of 0.084 (0.189 at a threshold of 0, rising to 0.271 at 0.5 and flat thereafter at 0.273): a permissive threshold that lets every edge propagate produces a noisier target than one that requires meaningful coupling, and the ordering stabilizes once the cutoff reaches 0.5. Feature normalization spans 0.037 — robust scaling at 0.234 against 0.197 for both min-max and z -score. That is smaller than the threshold spread but of the same order, not negligible beside it: rank-based robust scaling and the two magnitude-preserving alternatives do not induce the same ordering, and the choice is a reported parameter of the scorer rather than an implementation detail. Neither spread indicates that the ordering is *correct*; they indicate only that it is stable under the free parameters of the oracle and the scorer, which is the weaker property a sensitivity sweep can establish.

S4. AHP Pairwise-Comparison Matrices and Their Consistency

Several weight vectors in this framework are derived by the Analytic Hierarchy Process. We state the two that back reported quantities, and we report a property of the matrix family that bears on how much the accompanying consistency ratios are worth.

A caveat on consistency ratios. Saaty’s consistency ratio detects *inconsistent* judgment; it does not detect a matrix constructed from its own answer. If a priority vector w is chosen first and entry (i, j) is then filled in as w_i/w_j , the resulting matrix is perfectly consistent, $CR \approx 0$, and the statistic certifies nothing. Such a matrix is rank-one: every row is a scalar multiple of every other. Writing $\sigma_1 \geq \sigma_2$ for its two largest singular values, the ratio σ_2/σ_1 separates the two cases — it is near zero for a back-filled matrix and appreciably positive for one carrying genuine judgment disagreement.

Table S2: Consistency diagnostics for the framework’s five AHP matrices. CR alone would suggest all five are sound; σ_2/σ_1 shows that three encode a declared priority vector rather than independent pairwise elicitation. The Availability matrix returns a slightly negative CR , which is not attainable for a genuinely elicited matrix ($\lambda_{\max} \geq n$) and arises here as floating-point noise around exact consistency.

Matrix	n	CR	σ_2/σ_1
Topic QoS (main manuscript, Eq. 3)	3	+0.0158	0.072
Fault Tolerance	3	+0.0029	0.027
Composite impact (I_{comp})	4	+0.0011	0.014
Maintainability	5	+0.0005	0.0006
Availability	5	−0.0029	0.0006

Only the Topic QoS matrix carries a consistency ratio that means what a consistency ratio is supposed to mean, and it is the one matrix a continuous-integration test guards on both sides: `tests/test_ahp_shrinkage.py` asserts $0.005 < CR < 0.10$, the lower bound existing precisely as a non-degeneracy check. We report the remaining vectors as declared constants with documented structure rather than as elicited judgment, and

note that this reading costs the framework little: the global sensitivity analysis (Section S1) finds eight of the ten weight constants to have $\mu^* \leq 0.023$, so the ranking results in Section 7 of the main manuscript do not rest on these values being optimal.

Table S3: Saaty matrix over the three Topic QoS dimensions backing $\text{QoS}(t)$ in Equation (3) of the main manuscript. Geometric-mean priority vector (0.2385, 0.6250, 0.1365); $\lambda_{\max} = 3.018$, $CI = 0.0092$, $RI = 0.58$, $CR = 0.0158$. The shipped constants (0.24, 0.62, 0.14) round this within 0.005.

	Reliability	Durability	Priority
Reliability	1	1/3	2
Durability	3	1	4
Priority	1/2	1/4	1

Table S4: Matrix over the four composite-impact criteria — reachability loss (RL), fragmentation (FR), throughput loss (TL), flow disruption (FD) — backing $I_{\text{comp}}(v)$ (Section 4.3 of the main manuscript). Raw priority vector (0.389, 0.255, 0.255, 0.100); after the framework’s $\lambda = 0.7$ shrinkage toward a uniform prior, (0.347, 0.254, 0.254, 0.145), which the shipped (0.35, 0.25, 0.25, 0.15) round. Per Table S2 this matrix is rank-one, so it documents the provenance of those constants without independently justifying them.

	RL	FR	TL	FD
RL	1	3/2	3/2	4
FR	2/3	1	1	5/2
TL	2/3	1	1	5/2
FD	1/4	2/5	2/5	1

S5. Generative Parameters of the Synthetic Corpus

Table S5 records the configuration behind each synthetic scenario of Table 4 in the main manuscript. It is replication detail rather than a result: the corpus shape a reader needs to interpret the evaluation is in Table 4 itself, and every configuration is committed in the replication package.

Table S5: Generative parameters of the twelve synthetic scenarios (eleven evaluation scenarios plus the ATM case study). Seed and fan-out figures are read from the committed configurations; per-entity counts are in Table 4 of the main manuscript. The modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them, computed from the committed topology rather than the config’s declared QoS targets, which domain-driven assignment does not always realize (Section 6.1 of the main manuscript).

Scenario	Seed	Pub	Sub	Modal QoS (R/D/P)
Air Traffic Management (ATM)	42	1.2	2.9	RELIABLE/VOLATILE/HIGH (41–81%)
Autonomous Vehicle (AV)	1001	2.5	5.0	RELIABLE/TRANSIENT_LOCAL/HIGH (45–80%)
Enterprise Pub-Sub	7007	3.0	4.5	RELIABLE/TRANSIENT_LOCAL/MEDIUM (29–79%)
Financial Trading	3003	4.0	6.0	RELIABLE/PERSISTENT/HIGH (40–89%)
Healthcare Integration	4004	2.5	3.0	RELIABLE/PERSISTENT/HIGH (40–88%)
Hub-and-Spoke	5005	2.0	7.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (33–67%)
Industrial SCADA	1902	1.1	1.0	RELIABLE/TRANSIENT/HIGH (31–80%)
IoT Smart City	2002	2.0	1.5	BEST_EFFORT/VOLATILE/LOW (56–75%)
Logistics Fleet	2104	1.8	1.9	RELIABLE/TRANSIENT_LOCAL/MEDIUM (40–76%)
Microservices Mesh	6006	1.5	2.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (40–60%)
Real-Time Gaming	2003	2.6	2.8	BEST_EFFORT/VOLATILE/HIGH (39–71%)
Telecom RAN	1801	2.2	1.6	RELIABLE/VOLATILE/HIGH (36–67%)

S6. Anti-Pattern Detection and Node-Type Stratification

Validated against $I_{\text{comp}}(v)$, the rule-based anti-pattern catalog reaches mean precision 0.244 and recall 0.915 ($F_1 = 0.386$) — but it implicates 93.8% of scored components, so that recall is close to what indiscriminate flagging achieves and the precision is near the base rate. The ATM scaling sweep (29 to 444 components, five seeds, seed-invariant) shows the mechanism: Cohen’s κ ends at -0.018 on the 444-component instance, against $+0.035$ at 29 components, with a flag rate of 92.3% at the largest scale. The path is not monotone — κ rises to $+0.086$ at 148 components before collapsing — so we do not claim a clean decay law, only the endpoint: at the largest scale the catalog’s agreement with the oracle is indistinguishable from chance, and marginally on the wrong side of it. The catalog does not become wrong at scale so much as it stops discriminating. We report it as a characterization of the catalog rather than a working triage mechanism, and delegate critical-set identification to the continuous rankers of Sections 7.1–7.2 of the main manuscript. A nineteenth detector, `DEEP_PIPELINE`, is excluded for path-combinatorial explosion (247,761 paths on a 29-component fixture).

Stratification vs. Pooling: Measured against the composite oracle $I_{\text{comp}}(v)$ over the eight scenarios of the detection benchmark, stratified RM rank correlations are $\rho = 0.566$ (Application, 8 scenarios), 0.119 (Broker, 6 scenarios), and 0.244 (Node, 8 scenarios), while pooled correlation across all types collapses to $\rho = 0.098$. Pooling node types triggers Simpson’s paradox by conflating disparate structural base rates across architectural layers: the pooled figure sits below every per-type figure it aggregates. This is why every evaluation in this paper is reported on a single stratum, and why pooled critical-set numbers should be read as inflated wherever they appear (Section 7.4 of the main manuscript). On that same pooled benchmark, unweighted degree centrality attains $\rho = 0.168$ and $F_1 = 0.442$ against RM’s 0.098 and 0.342 — RM is beaten by degree on both, confirming that it serves as an attribution instrument rather than an unconstrained global ranker.

S7. Real-World Evaluation of the Explanation Layer

We evaluated the framework on five open-source distributed systems transcribed from their public repositories by dedicated architectural adapters, with results presented in Table S6. We note the scope of this evaluation before reporting it. Online Boutique is a vendor demonstration application and Train-Ticket an academic benchmark, Home Assistant is an authentic open-source IoT automation platform, and EdgeX Foundry is an industrial edge computing reference implementation; each is an abstraction of its upstream repository rather than a complete transcription; and their labels come from the same simulation oracle used throughout, so what follows tests topological generalization, not agreement with observed field failures. Two further limits bound what this table can support. First, the predictor scored in Table S6 is the deterministic explanation layer $Q(v)$ together with the two training-free topological baselines; the learned model is evaluated separately in Section 7.4.1 of the main manuscript. Both are scored against $I^*(v)$, but under different injector settings (see the caption), so the two tables are not commensurable. Second, the labels are strongly tied at zero in several architectures — 13 of 32 Applications in Autoware, 14 of 22 in Cloud Microservices, 27 of 41 in Train-Ticket, and 12 of 22 in EdgeX carry zero simulated impact (the complement of the “Impactful Apps” column), whereas Home Assistant exhibits much lower zero-inflation with 17 of 24 applications actively participating in failure cascades. Spearman ρ over populations that are heavily tied is dominated by how those ties are handled; the values below use midrank ties throughout.

Key Insights for RQ4:

1. **The explanation layer ranks unseen architectures well, but this is not a learned-transfer result.** RM / $Q(v)$ attains $\rho = 0.800$ on EdgeX Foundry, 0.778 on Cloud Microservices, 0.759 on Train-Ticket, 0.685 on Autoware.universe, and 0.514 on Home Assistant. Because $Q(v)$ is a closed-form scoring function rather than a fitted model, applying it to a new architecture involves no transfer in the inductive sense of Section 7.2 of the main manuscript: nothing was trained, and no distribution shift is being survived. What this table establishes is that the deterministic attribution of Section 5 of the main manuscript remains informative on architectures we did not generate. The learned model is a separate question, answered in Section 7.4.1 of the main manuscript.

Table S6: Open-source reference systems scored by the deterministic explanation layer RM / $Q(v)$ — no GNN checkpoint is invoked, so $\pm$ is oracle variance over five simulation seeds, not training variance. “Gain vs. Deg” is $\rho_Q - \rho_{\text{deg}}$. Ground truth here is $I^*(v)$ (**FaultInjector**) — the same oracle as Table 9b of the main manuscript, but run with the validation CLI’s cascade-depth cap of 5 and averaged over five injector repeats per node, where Table 9b’s labels run to unlimited depth. Simulator-derived labels, not observed incidents.

Real-World Architecture	$ V $	$ V_{\text{app}} $	Spearman ρ	Kendall τ	App $F_1@K$	Pooled $F_1@K$	Impactful Apps	Gain vs. Deg
Autoware.universe (ROS 2)	75	32	0.685 $\pm$ 0.010	0.513	0.333	0.800	19 / 32	+0.357
Cloud Microservices Mesh	60	22	0.778 $\pm$ 0.001	0.639	0.500	1.000	8 / 22	+0.014
Train-Ticket Booking Mesh	90	41	0.759 $\pm$ 0.001	0.605	0.625	1.000	14 / 41	+0.264
Home Assistant (Smart Home)	63	24	0.514 $\pm$ 0.016	0.373	0.250	0.667	17 / 24	+0.289
EdgeX Foundry (Industrial IoT)	63	22	0.800 $\pm$ 0.015	0.563	0.000	1.000	10 / 22	+0.427

Table S6 is not commensurable with Table 7’s RM column ($\rho = 0.133$), but the cause is not the oracle: Table 7, Table 9b and Table S6 all score against $I^*(v)$ (**FaultInjector**). Table 7 is the synthetic LOSO corpus, whereas Table S6 and Table 9b both score the five transcribed systems on the Application stratum; under Table 9b’s unlimited-depth labels RM attains $\rho = 0.516$ on those same systems. What separates the figures is therefore a real-versus-generated difference together with the injector configuration recorded in the caption, with the tied-label structure described above contributing to both. The practical consequence is that no RM figure in this paper should be compared across tables without checking which population and which injector configuration produced it.

- Stratified vs. Pooled Critical Set Identification:** On the stratified Application population (V_{app} with $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$), $Q(v)$ identifies the top-20% critical services with $F_1@K = 0.333$ in Autoware, 0.500 in Cloud Microservices, 0.625 in Train-Ticket, 0.250 in Home Assistant, and 0.000 in EdgeX Foundry (where failure impact is dispersed broadly across active device services). When evaluated across the pooled multigraph ($K = \text{round}(0.20 \cdot |V|)$), the pooled $F_1@K$ reaches 0.800, 1.000, 1.000, 0.667, and 1.000, respectively. This disparity is the base-rate phenomenon of Section 6.3 of the main manuscript: the passive infrastructure entities (Topics, Nodes, Libraries) carry constant zero failure impact under the injector configuration used for these runs, so a pooled top- K set is largely populated by correctly identifying inert nodes. The pooled column is therefore inflated and we do not read it; the stratified V_{app} column is the operational figure.
- Framework Acceptance Gates and Cross-Domain Disparities:** The framework ships a four-condition release gate ($\rho \geq 0.75$, $\text{Overlap}@K \geq 0.70$, $\text{SPOF } F_1 \geq 0.60$, prediction gain ≥ 0.02). EdgeX Foundry passes all four gates across all five seeds (100% pass rate), driven by strong rank correlation ($\rho = 0.800$), pooled $F_1 = 1.00$, and robust articulation detection ($\text{SPOF } F_1 = 0.80$). In contrast, the pass rate is zero on the other four architectures: Autoware fails the ρ and SPOF conditions, Cloud Microservices fails SPOF and prediction gain, Train-Ticket fails SPOF, and Home Assistant misses the ρ gate ($\rho = 0.514$) despite scoring a perfect SPOF $F_1 = 1.00$. What this establishes is a negative result about thresholds, not a positive one about ordering: a gate calibrated on our synthetic corpus fires correctly on one architecture in five, so its absolute cut-offs are domain-specific and do not transfer as shipped. The ordering half of the question — whether the ranking underneath those thresholds transfers — is not answered here and is answered negatively in Section 7.4.1 of the main manuscript, once the population is restricted to components that actually propagate a failure.
- Predictive Advantage over Structural Heuristics:** The “Gain vs. Deg” column reports $Q(v)$ ’s rank-correlation margin over an unweighted degree centrality baseline (+0.427 on EdgeX Foundry, +0.357 on Autoware, +0.289 on Home Assistant, +0.264 on Train-Ticket, +0.014 on Cloud Microservices), so on this population and this oracle the hierarchical attribution of Section 5 of the main manuscript orders components better than degree centrality does. The margin does not generalize beyond those conditions, and we scope it accordingly: on the pooled synthetic detection benchmark of Section 7.3.6 of the main manuscript — which is measured against $I_{\text{comp}}(v)$ (**FailureSimulator**) rather than the $I^*(v)$ used here, and across all entity types — degree *beats* RM on both rank correlation (0.168 vs. 0.098) and $F_1@K$ (0.442 vs. 0.342). The two findings are not in conflict — one is stratified on Applications in five transcribed systems under $I^*(v)$, the other pooled across types in eight generated ones under $I_{\text{comp}}(v)$ — but neither supports a general claim that RM dominates degree. Wilcoxon signed-rank

tests over the five systems do not reach significance ($p \geq 0.33$), which at $n = 5$ they could hardly do; we report them for completeness rather than as support.

5. **The QoS-weighted heuristic cannot be scored here, and the reason is structural rather than incidental.** The unweighted Topo reference is computable ($\rho = 0.307/0.891/0.528$ on Autoware, Cloud Microservices and Train-Ticket; mean 0.511, Table 9b of the main manuscript); Topo-QoS is not. The obstacle is not missing QoS data — every topic in all five adapters carries declared `durability`, `reliability` and `transport_priority` profiles, and projecting $w(t)$ onto the structural edges yields non-unit weights on 45.3–62.2% of them. It is that Topo reads betweenness off the cached `DEPENDS_ON` projection, whereas the QoS-weighted variant must recompute it on the raw multigraph, where Applications never route messages and betweenness is identically zero for all of them — the same degeneracy Section 6.2.1 of the main manuscript gives as the reason topological baselines are run on the projection at all. We record the gap rather than substitute a number for it; the consequence, that the real-world margin is measured against the weaker training-free reference, is stated in Section 8.4 of the main manuscript. Separately, projection-based baselines assign constant zero to Applications carrying no derived `DEPENDS_ON` edge, accentuating sensitivity to tied ground truth.

S8. HGT Relational Attention Weight Analysis

Figure S2 visualizes the relational mutual attention distributions of a Heterogeneous Graph Transformer on the ATM case study architecture. *The model is not the predictor evaluated elsewhere in this paper.* It is a three-layer, four-head HGT trained for 100 epochs on the ATM scenario alone at seed 42, built by `reproduce/extract_attention.py` for this figure and discarded afterwards. The LOSO-evaluated checkpoints cannot stand in: they are trained over the full corpus, carry a 40-relation schema against this topology’s 32, include `Library` publish and subscribe relations the ATM graph does not contain, and are two layers deep rather than three. Nothing below therefore speaks to what the evaluated predictor attends to; it speaks to whether typed attention differentiates at all.

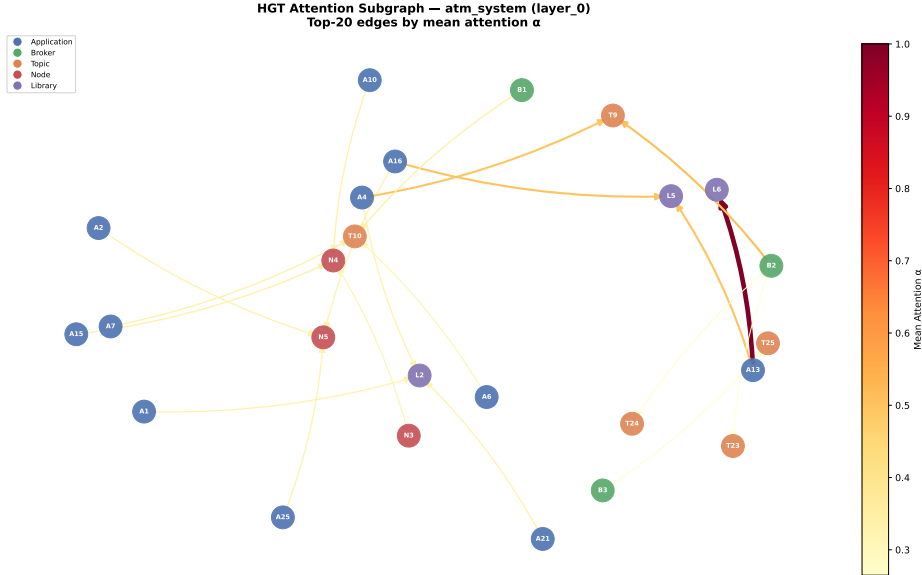


Figure S2: Relational attention on the ATM case study, from a three-layer HGT trained for 100 epochs on this scenario alone (seed 42) — not the predictor evaluated in Sections 7.1–7.4 of the main manuscript. Mean α spans 0.113–0.222 across the eight relation types and is governed substantially by destination in-degree. A qualitative illustration on one topology, not evidence of general relation prioritization.

Aggregated by relation type, first-layer mean attention across all eight relations runs: `USES` (App→Lib) 0.220; `ROUTES` 0.187; `SUBSCRIBES_TO` 0.172; `USES` (Lib→Lib) 0.169; `PUBLISHES_TO` 0.160; `RUNS_ON` (App→Node) 0.156; `CONNECTS_TO` 0.135; and `RUNS_ON` (Broker→Node) 0.113. Across the three layers no relation-type mean moves by more than 0.019, but the rank order is *not* stable: the four relations between 0.15 and 0.18 are spaced more closely than that drift, and they permute between layers. Three methodological cautions bound what this visualization supports:

1. The spread across all eight relation types is narrow (0.113–0.222, and only 0.156–0.220 among the five carrying more than nine edges), indicating a mild distributional tendency rather than a clear-cut structural separation.
2. Because α is computed as a softmax over each destination node’s incoming neighborhood $\mathcal{N}(v)$, the mean attention weight α is substantially governed by local in-degree. The single largest weight in the whole extraction ($\alpha_{uv} = 1.00$, on a `USES` edge into a library) lands on a destination of in-degree one, where $\alpha = 1.0$ holds by arithmetic and carries no information. The thinnest relations sit at the extremes of the ordering: `USES` (Lib→Lib) carries 3 edges and `RUNS_ON` (Broker→Node) carries 5, against 85 for `SUBSCRIBES_TO`.
3. The figure is a function of the corpus snapshot, not a stable property of the architecture. Regenerating the ATM topology changed its edge set without changing any entity count (`PUBLISHES_TO` 56 → 48, `SUBSCRIBES_TO` 90 → 85, `ROUTES` 36 → 37, `CONNECTS_TO` 7 → 9), and that alone moved `USES` (Lib→Lib) from 0.225 to 0.169 — from above `ROUTES` to below it. The extraction itself is deterministic, so the whole of that shift is input sensitivity on a 74-node graph.

Consequently, Figure S2 serves as an illustrative artifact confirming that multi-head heterogeneous attention remains active across relation types, rather than a statistical proof of general relational importance.

References

- [1] M. D. Morris, Factorial sampling plans for preliminary computational experiments, *Technometrics* 33 (2) (1991) 161–174.
- [2] F. Campolongo, J. Cariboni, A. Saltelli, An effective screening design for sensitivity analysis of large models, *Environmental Modelling & Software* 22 (10) (2007) 1509–1518.
- [3] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, S. Tarantola, *Global Sensitivity Analysis: The Primer*, John Wiley & Sons, 2008.
- [4] I. M. Sobol’, Sensitivity estimates for nonlinear mathematical models, *Mathematical Modelling and Computational Experiments* 1 (4) (1993) 407–414.
- [5] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [6] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: *Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS)*, 2018, pp. 287–296.
- [7] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software, <https://github.com/GoogleCloudPlatform/microservices-demo> (accessed 9 September 2026) (2024).
- [8] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [9] Home Assistant Community, Home assistant: Open source home automation that puts local control and privacy first, Software, <https://www.home-assistant.io/> (accessed 9 September 2026) (2024).
- [10] Linux Foundation LF Edge, Edgex foundry: An open, vendor-neutral edge iot middleware platform, Software, <https://www.edgexfoundry.org/> (accessed 9 September 2026) (2024).

S9. Convergent Validity Over Simulation Oracles

We evaluated inter-oracle agreement across $I^*(v)$ (**FaultInjector**), $I_{\text{comp}}(v)$ (**FailureSimulator**), and $I_{\text{dyn}}(v)$ (**MessageFlowSimulator**) over the twelve inductive folds of Table 4 of the main manuscript, summarized in Table S3.1:

Table S7: Inter-oracle agreement across simulation paradigms (chance top- K Jaccard is 0.111) over the twelve LOSO topologies, Application population. One comparison per topology: the five seeds enter I^* only, averaged into a single label per component, while I_{comp} and I_{dyn} each run once at seed 42. I_{dyn} denotes the queue-flow discrete-event simulation oracle, I^* denotes the graph topological cascade injection oracle, and I_{comp} denotes the multi-criteria composite oracle. ρ^+ restricts the correlation to components both oracles score non-zero, separating directional agreement from agreement on which components are harmless. I_{dyn} was measured under enforced QoS contracts at a calibrated per-subscriber operating point ($\rho_{\text{util}} = 0.65$); the run is reproducible from the commit its artifact names.

Oracle pair	Mean ρ (range)	Mean ρ^+	Mean τ	Jaccard@ K	Tie-robust
I_{dyn} vs. I^*	0.620 (0.290–0.924)	0.441	0.478	0.365	0.361
I_{comp} vs. I^*	0.395 (0.083–0.653)	0.353	0.290	0.266	0.261
I_{comp} vs. I_{dyn}	0.366 (0.069–0.564)	0.343	0.253	0.276	0.276

The queue-flow simulator and the topological cascade oracle agree substantially but not interchangeably ($\rho = 0.620$, Jaccard 0.365 against 0.111 expected by chance). **This is the reading the ceiling supports.** I^* ’s own seed-to-seed test–retest across these same twelve folds runs 0.811–1.000 (median 0.982; Section 7.1 of the main manuscript), so I_{dyn} agrees with I^* distinctly *less* closely than I^* agrees with itself. The gap is the point: a behavioural oracle that reproduced the topological one to within label noise would be re-measuring the topology rather than corroborating it. Ranking over discrete-event queueing traffic recovers most of the topological ordering while retaining content the cascade abstraction does not express.

Two boundaries qualify this. First, $\rho^+ = 0.441$ against $\rho = 0.620$ shows that a substantial share of the agreement is the two oracles concurring on which components are *harmless*; restricted to components both score non-zero, agreement is weak in five of twelve folds (Autonomous Vehicle 0.101, Microservices 0.099, Hub-and-Spoke 0.113). Second, Microservices is simultaneously the weakest fold ($\rho = 0.290$) and the least reproducible label in the corpus (test–retest 0.811, top- K Jaccard 0.720), so part of that deficit is label noise rather than construct divergence.

Both oracles have a floor, and only one of them had been measured. The reading above is stated against I^* ’s test–retest alone, which silently treats I_{dyn} as noiseless. It is not: it is a stochastic discrete-event simulation run here at a single seed. Re-running both non-primary oracles across seeds {42, 123, 456} on three folds spanning the corpus — ATM, Healthcare Integration, and Microservices Mesh, the last being the weakest fold above — gives I_{comp} a test–retest of 1.000 everywhere (its Bernoulli cascade draws are effectively deterministic at the shipped edge weights, so its single seed costs nothing), and I_{dyn} a test–retest of 0.958–0.972 on ATM, 0.828–0.867 on Healthcare, and 0.741–0.821 on Microservices. The subset is three folds rather than twelve because I_{dyn} costs one discrete-event run per component per seed; it is chosen to bracket the corpus rather than to average over it.

The consequence is specific. On Microservices, I_{dyn} agrees with *itself* at 0.741–0.821 — at or below I^* ’s own 0.811 on that same fold — so the $\rho = 0.290$ reported there is bounded by two dispersions, not one, and attributing the shortfall primarily to I^* ’s label noise overstates what can be separated. Elsewhere the 0.620 headline is less affected: on ATM and Healthcare I_{dyn} reproduces itself at 0.83–0.97, well above the cross-oracle agreement, so the construct-divergence reading of the gap survives on those folds. We report the floors rather than correcting the headline for them, because a noise-corrected correlation would require the twelve-fold multi-seed run this subset stands in for.

S10. In-Distribution Significance Tests

Paired tests over the twelve in-distribution scenarios of Table 5 of the main manuscript. They are reported here rather than in the body because Section 7.2 declines to read the in-distribution typed–untyped contrast

as evidence about typing: the typed pair consumes the native multigraph while the homogeneous pair consumes the Application–Library projection, so the margin mixes typed message passing with multi-entity visibility. The tests are given for completeness, not as support for a claim.

Table S8: Paired Wilcoxon signed-rank tests across in-distribution scenarios ($n = 12$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p -value	Significance
HGT-QoS vs. Topo	+0.291	11/12	2.0	0.0015	Significant ($p < 0.01$)
Topo-QoS vs. Topo	+0.198	12/12	0.0	0.0005	Significant ($p < 0.001$)
HGT-QoS vs. GAT-QoS	+0.222	11/12	3.0	0.0024	Significant ($p < 0.01$)
HGT-QoS vs. GAT	+0.118	9/12	21.0	0.1763	Not significant
HGT-QoS vs. Topo-QoS	+0.093	9/12	23.0	0.2334	Not significant
HGT vs. GAT	+0.082	8/12	29.0	0.4697	Not significant
HGT-QoS vs. HGT	+0.037	8/12	32.0	0.6221	Not significant
GAT-QoS vs. Topo-QoS	−0.129	4/12	21.0	0.1763	Not significant

S11. Typed Node Feature Schema

Both pathways read the same typed node properties from G_{analysis} : the predictive pathway (Section 4 of the main manuscript) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5 of the main manuscript) aggregates them into its quality profile. All five entity types share indices 0–17, a common block of topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load) produced by the deterministic analysis stage whose cost is characterized in Section 7.5 of the main manuscript. Type-specific features extend that block:

- **Application (23 dims)**: indices 18–22 add source-code metrics from SCA — lines of code, cyclomatic complexity, Martin’s instability $I_{\text{code}} = C_e/(C_a + C_e)$ [5], Lack of Cohesion in Methods, and the composite Code Quality Penalty (CQP).
- **Library (25 dims)**: the Application block plus two library-specific blast-radius drivers (23–24): the normalized size of the transitive reverse-USES closure, and the normalized count of distinct subscribers reachable from topics published within that closure.
- **Broker (19 dims)**: index 18 is normalized queue buffer capacity.
- **Topic (22 dims)**: indices 18–21 are publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node (20 dims)**: indices 18–19 are normalized CPU core allocation and physical memory.

S12. Corpus Subset Behind Each Analysis

Not every analysis in Section 7 of the main manuscript runs on the full corpus: the sensitivity sweeps and the detection benchmark predate the four extended domains and operate on smaller cached subsets. This table states which subset backs each reported figure.

S13. Per-Scenario Corpus Composition

Entity and edge counts for every scenario, read from the committed topology files rather than from the generator configurations. Continuous integration asserts that each dataset regenerates byte-identically from its configuration and that these counts match what is on disk.

Table S9: Which corpus subset backs each analysis. Figures from different rows are not directly comparable, and we do not compare them.

Analysis	Scenario subset	n	Reported in
In-distribution ranking	Eleven evaluation scenarios + ATM	12	Table 5, Supp. S10
Inductive LOSO	Eleven evaluation scenarios + ATM	12	Table 7, Sections 7.1–7.2
QoS edge-feature ablation	Same twelve LOSO folds	12	Section 7.3.1
Weight sweeps and Morris screening	Six–seven core synthetic domains	6–7	Supplementary S1
Cross-oracle convergent validity	Same twelve LOSO folds	12	Section 7.3.2, Supp. S9
Anti-pattern detection, stratification	Seven core domains + ATM	8	Section 7.3.3
Relational attention illustration	ATM case study alone	1	Supplementary S8
Real-world zero-shot transfer	Five open-source systems	5	Table 9b, Supp. S7

Table S10: Experimental evaluation corpus. The twelve synthetic topologies are the inductive Leave-One-Scenario-Out folds of Table 7 of the main manuscript; the five real-world systems are withheld from every training fold and used only for zero-shot transfer (Section 7.4 of the main manuscript). Counts are read from the committed topology files rather than from the generator configurations.

Dataset / Architecture	System Paradigm	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
<i>Synthetic evaluation scenarios (evaluation role in the manifest)</i>								
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	774
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,216
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	631
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	389
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	691
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,188
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	678
Telecom RAN	5G Radio Access Network	225	120	55	8	20	22	881
Industrial SCADA	Plant Control Telemetry	254	140	70	4	25	15	824
Real-Time Gaming	Multiplayer State Sync	158	75	38	5	12	28	630
Logistics Fleet	Vehicle Telematics Mesh	205	110	50	7	18	20	755
<i>Synthetic case study (LOSO fold; case_study role in the manifest)</i>								
Air Traffic Management (ATM)	ICAO Global ATM Concept	74	26	27	5	8	8	261
<i>Real-world reference systems (never used as training folds)</i>								
Autoware.universe [6]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [7]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [8]	Real-World Microservices	90	41	30	3	8	8	162
Home Assistant [9]	Real-World Smart Home	63	24	22	3	6	8	119
EdgeX Foundry [10]	Real-World Industrial IoT	63	22	24	3	6	8	112
Synthetic subtotal (12 LOSO folds)		2,461	1,321	615	65	202	258	10,918
Real-world subtotal (5 systems)		351	141	120	16	32	42	700
Total		2,812						11,618

S14. Running Example: Structural Graph and Its Projection

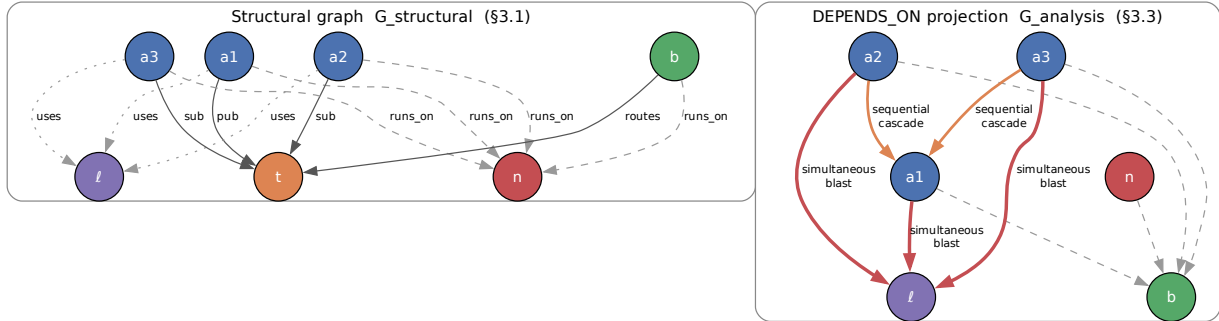


Figure S3: Running example: the raw structural graph (left) and the `DEPENDS_ON` projection derived from it (right). The projection makes implicit runtime dependencies explicit—a subscriber depends on the publishers of its topics even though no structural edge joins them—while the simulators continue to operate on the structural view alone.